

Package ‘breedRBayes’

September 3, 2026

Type Package

Title ASReml-Style Formula Front-End for the BGLR Bayesian Engine

Version 0.0.0.9000

Description A formula-based interface, inspired by 'asreml' and 'sommer', for fitting Bayesian mixed models with the 'BGLR' engine. Supports genomic prediction (GBLUP), random-regression / reaction-norm models, multi-trait models and factor-analytic structures. Provides extraction of posterior chains, variance components, heritabilities as full posterior distributions, and Markov-chain convergence diagnostics.

License MIT + file LICENSE

Encoding UTF-8

LazyData false

Depends R (>= 4.1.0)

Imports BGLR,
coda,
RhpcBLASctl,
stats,
utils,
ggplot2

Suggests SpATS,
EnvRtype,
parallel,
testthat (>= 3.0.0),
knitr,
rmarkdown

RoxygenNote 7.3.2

Roxygen list(markdown = TRUE)

Config/testthat/edition 3

Contents

as_mcmc [2](#)

bbglr	2
gebv	4
heritability	4
legendre_basis	5
mcmc_diag	6
parse_model	6
plot_posterior	7
plot_trace	8
solution	8
varcomp	9

as_mcmc	<i>Convert a fitted model's posterior draws to a coda object</i>
---------	--

Description

Reads the variance-component chains (and optionally the intercept mu) written by BGLR and returns them as a [coda::mcmc.list](#) (one component per chain), ready for [mcmc_diag\(\)](#) or coda/bayesplot functions.

Usage

```
as_mcmc(x, ...)  
  
## S3 method for class 'breedRB_fit'  
as_mcmc(x, what = "varcomp", ...)
```

Arguments

- x A breedRB_fit.
- ... Unused.
- what Which quantities to return: any of "varcomp" and "mu".

Value

A [coda::mcmc.list](#).

bbglr	<i>Fit a Bayesian mixed model with the BGLR engine using asreml-style formulas</i>
-------	--

Description

Fit a Bayesian mixed model with the BGLR engine using asreml-style formulas

Usage

```
bbglr(
  fixed,
  random = NULL,
  residual = NULL,
  data,
  relmat = list(),
  nIter = 5000,
  burnIn = 1000,
  thin = 5,
  nChains = 1,
  seed = 123,
  saveAt = NULL,
  verbose = TRUE
)
```

Arguments

fixed	Two-sided formula for the fixed/mean part, e.g. <code>yield ~ 1 + env</code> . Use <code>cbind(t1, t2) ~ ...</code> for a multi-trait model.
random	One-sided formula of random terms, e.g. <code>~ vm(gen, G)</code> . Special functions: <code>vm(f, K)</code> genomic effect with relationship matrix <code>K</code> ; <code>leg(x, n) / rr(x, n)</code> random regression of order <code>n</code> ; <code>fa(f, k) / rrc(f, k)</code> factor-analytic. Bare names are factors; <code>a:b</code> is an interaction.
residual	One-sided residual formula. <code>NULL/~ units</code> = homogeneous; <code>~ dsum(~units env)</code> = heterogeneous residual variances by <code>env</code> .
data	A data frame.
relmat	Named list of relationship / covariance matrices referenced by <code>vm()</code> (e.g. <code>list(G = kinship)</code>). Matrices use covariance (<code>K</code>), not its inverse. Row/column names must match the factor levels.
nIter, burnIn, thin	MCMC controls passed to BGLR.
nChains	Integer number of independent chains (distinct seeds). Enables Gelman-Rubin diagnostics when <code>> 1</code> .
seed	Base random seed; chain <code>c</code> uses <code>seed + c - 1</code> .
saveAt	Directory for BGLR binary output. Defaults to a fresh temp dir.
verbose	Logical; print BGLR progress.

Value

An object of class `breedRB_fit`.

Examples

```
G <- readRDS(system.file("extdata", "kinship.matrix.rds", package = "breedRBayes"))
dat <- read.csv(system.file("extdata", "data_soy.csv", package = "breedRBayes"))
fit <- bbglr(yield ~ 1 + env, random = ~ vm(gen, G), data = dat,
             relmat = list(G = G), nIter = 2000, burnIn = 500, nChains = 2)
```

gebv

Posterior genomic estimated breeding values (GEBV)

Description

Deprecated. Use `solution()`, which extracts posterior solutions for any term (genomic `vm()` BLUPs, plain random-factor BLUPs, and fixed effects). `gebv(fit, term)` is equivalent to `solution(fit, term, type = "random")` with the value column renamed `gebv`.

It still works for a `vm()` genomic term for now, mapping the ridge coefficients back through the PC rotation (`genetic value = PC %*% b`).

Usage

```
gebv(fit, term = NULL, prob = 0.95)
```

Arguments

<code>fit</code>	A <code>breedRB_fit</code> (single-trait).
<code>term</code>	Genetic term identifier (label or key). Defaults to the first <code>vm()</code> term. For a non-genomic random effect use <code>solution()</code> instead.
<code>prob</code>	Central credible-interval mass (default 0.95).

Value

A data frame with ID, posterior `gebv` (mean), `sd`, `lower`, `upper`, ordered by decreasing `gebv`, with the pooled draws matrix (`nDraws` x `nLevel`) attached as attribute `"draws"`.

See Also

`solution()`, the general replacement.

Examples

```
head(gebv(fit))
```

heritability

Posterior distribution of heritability

Description

Computes narrow-sense (genomic) heritability for every MCMC draw, returning the full posterior distribution rather than a single point estimate:

$$h^2 = \sigma_{genetic}^2 / (\sigma_{genetic}^2 + \sum \sigma_{other}^2 + \sigma_e^2).$$

Usage

```
heritability(fit, genetic = NULL, denominator = NULL, prob = 0.95)
```

Arguments

fit	A <code>breedRB_fit</code> .
genetic	Character vector of genetic term identifiers (label or key) forming the numerator. Defaults to the <code>vm()</code> term(s).
denominator	Character vector of term identifiers summed in the denominator alongside <code>varE</code> . Defaults to all random terms.
prob	Central credible-interval mass (default 0.95).

Value

A list of class `breedRB_h2`: `summary` (mean/median/sd/CI) and `draws` (posterior vector of h^2).

Examples

```
h2 <- heritability(fit); h2$summary; hist(h2$draws)
```

legendre_basis

Orthogonal / orthonormal Legendre polynomial basis on [-1, 1]

Description

Constructs a Legendre polynomial basis evaluated at a numeric vector scaled to `[-1, 1]`, up to a specified order, using the three-term recurrence. Optionally returns the orthonormal basis.

Usage

```
legendre_basis(x, order, orthonormal = FALSE)
```

Arguments

x	Numeric vector (already scaled to [-1, 1]) where the basis is evaluated.
order	Integer. Maximum polynomial degree. Returns columns for degrees 0..order.
orthonormal	Logical. If TRUE, applies orthonormal scaling to each degree.

Value

A numeric matrix $\text{length}(x) \times (\text{order} + 1)$; column j is the degree $j - 1$ polynomial.

Examples

```
legendre_basis(seq(-1, 1, length.out = 5), order = 3)
```

mcmc_diag	<i>Markov-chain convergence diagnostics for a fitted model</i>
-----------	--

Description

Summarises convergence of the variance-component chains: effective sample size, Geweke's z , and (when the fit has >1 chain) the Gelman-Rubin potential-scale-reduction factor (R -hat).

Usage

```
mcmc_diag(fit, what = "varcomp", plot = FALSE)
```

Arguments

fit	A <code>breedRB_fit</code> .
what	Passed to <code>as_mcmc()</code> (default "varcomp"; add "mu" to include the intercept).
plot	Logical. If TRUE, also render <code>ggplot2</code> trace plots of the monitored chains (via <code>plot_trace()</code>) and attach the plot object to the returned data frame as the "plot" attribute. Default FALSE.

Value

A data frame with one row per monitored parameter: `param`, `n_eff`, `geweke_z`, and `Rhat` (NA for single-chain fits). When `plot = TRUE`, the corresponding `ggplot` is attached as `attr(, "plot")`.

Examples

```
mcmc_diag(fit)
d <- mcmc_diag(fit, plot = TRUE) # prints trace plots
attr(d, "plot")                 # the ggplot object
```

parse_model	<i>Parse asreml-style fixed / random / residual formulas</i>
-------------	--

Description

Turns the three model formulas into an internal, engine-agnostic description used by `bbg1r()`. Not usually called directly.

Usage

```
parse_model(fixed, random = NULL, residual = NULL)
```

Arguments

fixed	Two-sided formula for the mean/fixed part, e.g. <code>yield ~ 1 + env</code> . Multi-trait models use <code>cbind(t1, t2) ~</code>
random	One-sided formula for random terms, e.g. <code>~ vm(gen, G) + env:vm(gen, G)</code> . NULL for none.
residual	One-sided residual formula. NULL (default) or <code>~ units</code> gives homogeneous residuals; <code>~ dsum(~units env)</code> gives heterogeneous residual variances by env.

Value

A `breedRB_terms` list: `response`, `fixed`, `random`, `residual`.

Examples

```
parse_model(yield ~ 1 + env, ~ vm(gen, G))
```

plot_posterior	<i>Posterior density plot of variance components or heritability</i>
----------------	--

Description

Posterior density plot of variance components or heritability

Usage

```
plot_posterior(x, ...)

## S3 method for class 'breedRB_fit'
plot_posterior(x, ...)

## S3 method for class 'breedRB_h2'
plot_posterior(x, ...)
```

Arguments

`x` A `breedRB_fit` (variance components) or a `breedRB_h2` object.
`...` Unused.

Value

A `ggplot` object.

<code>plot_trace</code>	<i>Trace plots of the variance-component chains</i>
-------------------------	---

Description

Trace plots of the variance-component chains

Usage

```
plot_trace(fit, what = "varcomp")
```

Arguments

`fit` A `breedRB_fit`.
`what` Passed to `as_mcmc()`.

Value

A `ggplot` object (iteration on x, value on y, one panel per parameter, coloured by chain).

<code>solution</code>	<i>Posterior solutions (BLUPs / fixed effects) for a model term</i>
-----------------------	---

Description

Extracts the posterior distribution of the effect solutions for any term in a fitted model, pooled across chains:

- **Random terms** return BLUPs. A `vm()` genomic term is back-mapped through its principal-component rotation to per-genotype values (identical to `gebv()`); a plain random factor (fitted without a relationship matrix) returns one solution per level; random regression / covariate terms return one solution per basis coefficient.
- **Fixed terms** return the posterior of each estimable coefficient (treatment contrasts; the reference level is the implicit baseline at 0).

Usage

```
solution(fit, term = NULL, type = NULL, prob = 0.95, add_mu = FALSE)
```


Arguments

<code>fit</code>	A <code>breedRB_fit</code> (single-trait).
<code>term</code>	Term identifier (the label as written in the formula, e.g. "gen" or "env", or the internal key). If NULL, the first random term is used.
<code>type</code>	Optional; one of "random" or "fixed". When supplied it is validated against the term's actual role (a mismatch is an error), which is a convenient guard when a name could be ambiguous.
<code>prob</code>	Central credible-interval mass (default 0.95).
<code>add_mu</code>	Logical (default FALSE). If TRUE, the model intercept <code>mu</code> is added to every draw, shifting the solutions onto the overall-mean scale (e.g. BLUP + <code>mu</code>). Most useful for random BLUPs; the addition is done per posterior draw so the credible intervals account for uncertainty in <code>mu</code> .

Value

A data frame with `effect` (level / coefficient name), `solution` (posterior mean), `sd`, `lower`, `upper`. Random-term rows are ordered by decreasing `solution`; fixed-term rows keep design order. The pooled draws matrix (`nDraws` x `nEffect`) is attached as attribute "draws", and the resolved role as attribute "type".

See Also

[gebv\(\)](#) for genomic breeding values from a `vm()` term.

Examples

```
solution(fit, term = "gen", type = "random")           # random BLUPs (no G matrix)
solution(fit, term = "gen", type = "random", add_mu = TRUE) # BLUP + mu
solution(fit, term = "env", type = "fixed")           # fixed-effect estimates
```

varcomp

Posterior variance components of a fitted model

Description

Returns the posterior distribution and summaries of every random-term variance and the residual variance, pooling all chains.

Usage

```
varcomp(fit, prob = 0.95, draws = FALSE)
```

Arguments

fit	A breedRB_fit.
prob	Central credible-interval mass (default 0.95).
draws	Logical; if TRUE also return the pooled posterior draws matrix.

Value

A data frame of per-component summaries (term, mean, median, sd, lower, upper), with the pooled draws attached as attribute "draws" when draws = TRUE.

Examples

```
varcomp(fit)
```